

Helix OTA 1.0.0-MVP — Test Strategy

Field	Value
Revision	2
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	The four-layer test strategy for the Helix OTA 1.0.0-MVP, derived from HelixConstitution §1 (source-presence gate, artifact gate, runtime/integration) and §1.1 (mutation meta-test). Maps each test layer to every MVP component (server, artifact-validator, signing, rollout [deferred], Android agent, database, API) and specifies unit, integration, contract, e2e (on the Orange Pi 5 Max), security, and load testing, plus a mutation-testing meta-test. Anchored to the already-performed physical validation recorded in VALIDATION_EVIDENCE.md.
Issues	The six NEW ota-* submodules do not yet exist as repos, so no test code has been written or executed against them; all per-module unit/integration/mutation claims are forward-looking test <i>plans</i>, not executed results (UNVERIFIED until the repos and CI exist). On-board e2e on the Orange Pi 5 Max / RK3588 has not been run; the board's AOSP A/B + Virtual A/B + AVB enablement is itself UNVERIFIED and is the top-priority hardware gate carried from integration_guide.md. (The HelixConstitution clause numbers §1, §1.1, §7.1, §11.4.6, §11.4.61, §11.4.123 are now VERIFIED against the authoritative constitution text at constitution/Constitution.md — see §2 and §13; this resolves the prior-revision UNVERIFIED tag on the clause numbers.)
Issues summary	

Field	Value
	The only <i>executed</i> validation to date is the artifact gate captured in <u>VALIDATION_EVIDENCE.md</u> (OpenAPI redocly-valid, SQL migrations applied up+down on live Postgres, k8s kubeconform-valid). Everything else here is a plan to be implemented in CI and on hardware. Rev 2: verified the six cited HelixConstitution clause numbers (§1, §1.1, §7.1, §11.4.6, §11.4.61, §11.4.123) against the live <u>Constitution.md</u> and removed the blanket “clause numbers UNVERIFIED” tag — the numbers are now confirmed FACT (§13). Rev 1: initial revision. Stand up the six NEW ota-* repos and wire a CI matrix that runs: the §1 artifact gate (redocly, Postgres up/down migration service, kubeconform, docker compose config) exactly as in <u>VALIDATION_EVIDENCE.md</u>; Go unit (go test) + testcontainers-go integration; Kotlin/KMP unit; OpenAPI contract conformance; mutation testing (go-mutesting / gremlins for Go, pitest for Kotlin/JVM). Execute the on-board e2e + corrupt-slot A/B rollback runbook on the Orange Pi 5 Max once the board’s A/B/VAB/AVB enablement is confirmed. Compile the Kotlin snippets once the AOSP/Android-SDK toolchain is wired (carried from <u>VALIDATION_EVIDENCE.md</u> Issues).
Fixed	
Continuation	

Table of contents

1. Purpose and scope
2. The four test layers (HelixConstitution §1 / §1.1)
3. Components under test
4. Layer × component matrix
5. Unit testing
6. Integration testing
7. Contract testing (OpenAPI conformance)
8. End-to-end on the Orange Pi 5 Max
9. Security testing
10. Load testing
11. Mutation testing (meta-test, §1.1)

12. Already-performed validation (evidence)

13. Anti-bluff notes

The metadata-table + table-of-contents requirement is mandated by HelixConstitution §11.4.61 (“Mandatory Markdown metadata table + structured-doc ToC”, User mandate 2026-05-19 — VERIFIED at Constitution.md line ~5687). This document carries its metadata table first, then its ToC immediately after, per that clause and documentation_standards.md §3.

1. Purpose and scope

This document defines the **test strategy** for the Helix OTA **1.0.0-MVP** release. It is normative for how the MVP is verified before it is considered done. It binds the four-layer testing model mandated by HelixConstitution §1 (source-presence gate, artifact gate, runtime/integration) and §1.1 (mutation meta-test) — clause numbers VERIFIED against the live Constitution.md (see §2) — to each MVP component, and it specifies the concrete test types (unit, integration, contract, e2e, security, load) plus the mutation meta-test.

The strategy is **catalogue-first** (see submodule_reuse_map.md): new test code is written only for the **six NEW** ota-* submodules (ota-protocol, ota-artifact-validator, ota-rollout-engine, ota-update-engine-bridge, ota-android-agent, ota-telemetry-schema). Behavior that lives in catalogue submodules (auth, security, database, Storage, observability, Herald, eventbus, ratelimiter, middleware, http3, mdns, recovery, config, discovery, cache; KMP: Auth-KMP, Security-KMP, Storage-KMP, Config-KMP) is the responsibility of those submodules’ own suites; Helix OTA tests verify only the **wiring and contracts** at the boundary, not the reused internals.

The **rollout engine** (ota-rollout-engine) is **deferred** to 1.0.1-staged-rollout (see 1.0.1-staged-rollout/README.md); its tests are specified here for completeness but are **out of scope for MVP execution** and marked (**deferred**) throughout.

Only the executed artifact-gate validation in VALIDATION_EVIDENCE.md is a confirmed result; everything else in this document is a **test plan**, not an executed outcome (see §13).

2. The four test layers (HelixConstitution §1 / §1.1)

The four layers below are a **direct map** of HelixConstitution §1 (“Test coverage is mandatory for every change”) — VERIFIED at Constitution.md line ~133. §1’s own four-row invariant table reads, verbatim: (i) “*The change is present in source*” → source-presence gate; (ii) “*The change survives compilation / packaging ... inspects the produced artifact ... verifies the change actually shipped*” → artifact gate; (iii) “*The change behaves correctly at runtime ... runtime / integration / on-device test*” → runtime/ integration; (iv) “*The gate itself is not bluffing → meta-test (mutation) entry that mutates the asserted condition and proves the gate catches the break*” → the mutation meta-test, further specified by §1.1 (“False-positive immunity is an invariant”, VERIFIED line ~147). The clause numbers are therefore confirmed FACT, not assumptions.

The strategy implements those four layers:

1. **Source-presence gate (§1).** Before anything is claimed to exist, the *source* that implements it must be present and identifiable. For MVP this means: the component’s source module exists, compiles/parses, and the artifact it produces is checked into the corpus (e.g. the OpenAPI spec, the SQL migrations, the k8s manifests, the docker-compose file, the Kotlin snippets). A claim with no backing source is a §1 violation.
2. **Artifact gate (§1).** Every produced artifact that is parseable or executable **MUST** be validated by a *real* tool, never by inspection alone. This is the gate already exercised in VALIDATION_EVIDENCE.md (redocly for OpenAPI, live-Postgres apply for SQL, kubeconform for k8s). The MVP CI re-runs these exact validators as the standing artifact gate.
3. **Runtime / integration (§1).** The code must actually run against real dependencies: Go services against real Postgres + MinIO via testcontainers-go and net/http/httptest; the Android agent against a host/fake update_engine bridge with a real ZIP_STORED payload; and the on-board e2e on the Orange Pi 5 Max.
4. **Mutation meta-test (§1.1).** A meta-test that *tests the tests*: seed faults (mutants) into the code under test and confirm the suite kills them. This guards against vacuous or assertion-free tests that pass without exercising behavior. §1.1 is explicit: “Every new gate **MUST** be paired with a mutation entry ... (1) Temporarily breaks the assertion ... (3) Asserts the gate now reports FAIL ... If the mutation round does not turn PASS → FAIL, the gate is a sham and must be rewritten.” The constitution also carries **Appendix A — Mutation testing — academic and industrial foundations** (VERIFIED, line ~8994). See §11.

These layers are cumulative: a component is not “done” until its source is present (§1), its artifacts pass the gate (§1), it runs in integration (§1), and its suite survives the mutation meta-test (§1.1).

3. Components under test

The MVP components and their owning module(s) (from submodule reuse map.md §3–§4):

Component	Primary module(s)	Language	Notes
Server (control plane / API host)	NEW ota-protocol + catalogue http3, middleware, ratelimiter, recovery, auth, security, database, Storage, observability, Herald, eventbus, config	Go	Wires reused submodules; owns route/policy wiring + handlers.
		Go	

Component	Primary module(s)	Language	Notes
Artifact-validator	NEW ota-artifact-validator (+ security, ota-protocol)		Structure → hash → signature → version monotonicity → target compatibility (artifact_validation.md).
Signing	catalogue security / Security-KMP (crypto primitives) + the OTA package signing in the Go build pipeline	Go + build tooling	SHA-256 + signature + AVB for MVP; TUF device-side deferred (ADR-0002 , signing_verification.md).
Rollout <i>(deferred)</i>	NEW ota-rollout-engine (+ database)	Go	Staged %, halt/advance. Deferred to 1.0.1 — tests planned, not run in MVP.
Android agent	NEW ota-android-agent (KMP) + ota-update-engine-bridge + ota-protocol + ota-telemetry-schema; KMP catalogue Auth-KMP, Security-KMP, Storage-KMP, Config-KMP	Kotlin / KMP	register → poll → download → verify → apply → report (integration_guide.md).
DB	catalogue database + the MVP migrations	SQL (PostgreSQL)	001_initial_schema.{up,down}.sql; schema in schema.md .
API	NEW ota-protocol (wire contracts) + the OpenAPI spec	OpenAPI 3.1 + Go	openapi.yaml , 12 paths / 24 schemas (per VALIDATION_EVIDENCE.md §1).

4. Layer × component matrix

✓ = in MVP scope; (def) = deferred to 1.0.1; – = not applicable; (evi) = already executed, see [VALIDATION_EVIDENCE.md](#).

Component	§1 source- presence	§1 artifact gate	Unit	Integration	Contract	e2e (board)	Security
Server	✓	✓	✓	✓	✓	✓	✓
Artifact- validator	✓	✓	✓	✓	—	✓	✓
Signing	✓	✓	✓	✓	—	✓	✓
Rollout	(def)	(def)	(def)	(def)	—	(def)	(def)
Android agent	✓	✓ (Kotlin compile UNVERIFIED)	✓	✓	✓ (consumer)	✓	✓
DB	✓	✓ (evi)	✓	✓	—	—	✓
API	✓	✓ (evi)	—	✓	✓	✓	✓

DB and API have no meaningful “unit”/“mutation” target of their own (the DB is DDL; the API is a declarative contract); their correctness is covered by the artifact gate ((evi)), contract conformance, and the integration/security/load layers that exercise them.

5. Unit testing

5.1 Go (server, artifact-validator, signing, rollout [deferred])

- **Framework:** standard `go test` with **testify** (require/assert) and **table-driven tests** as the default style. Each table row names a case, inputs, and expected outcome.
- **ota-protocol** — round-trip encode/decode of wire types, manifest schema, status/event enums; table tests over valid + malformed payloads. Pure contracts, no I/O, fully deterministic.
- **ota-artifact-validator** — table tests driving the pipeline stages on **raw artifact bytes** (the module is independently testable on bytes per [submodule_reuse_map.md §4](#)):
 - structure: well-formed vs truncated/garbage zip;
 - hash: matching vs mismatched SHA-256;
 - signature: valid vs forged/absent signature (crypto primitive supplied by security);
 - version monotonicity: newer accepted, equal/older rejected;
 - target compatibility: matching vs mismatching device/build target.
- **Signing** — table tests over verify-good / verify-forged / verify-tampered using the security primitives; the Go-side OTA package signing helpers (the `ota_from_target_files` / `--package_key` / `--payload_signer` wrapping described in [build_integration.md](#)) get unit coverage on argument assembly and error paths.
- **Server handlers** — `net/http/httptest` exercises each handler in isolation with a fake store/port; table tests over auth-required vs anonymous, valid vs invalid bodies, and the error-mapping paths. (Reused submodule internals are *not* re-tested here.)

- **ota-rollout-engine** (*deferred*) — table tests with a **fake clock + fake telemetry** (per its decoupled boundary): phase advance, halt-on-failure-signal, and no-HTTP invariants.

5.2 Kotlin / KMP (Android agent)

- **Framework:** Kotlin unit tests (JUnit/kotlin.test on the JVM target where the logic is platform-independent). Logic that does not touch Android APIs is tested in `commonTest`.
- **ota-android-agent** — duty-cycle state machine (register → poll → download → verify → apply → report), poll-cadence (15 min + jitter) and backoff computed against a **test driver / fake clock** ([integration_guide.md §runtime/integration](#)), error-code mapping from the engine callbacks into telemetry events.
- **ota-telemetry-schema** — encode/decode round-trips of telemetry events/metrics shared by server + agent.
- **ota-update-engine-bridge** — thin Android-only surface; the host-testable portion (argument assembly for `applyPayload(url, offset, size, props)` and the four header props `FILE_HASH/FILE_SIZE/METADATA_HASH/METADATA_SIZE` per [update_engine_integration.md](#)) is unit tested; the actual HAL call is covered in `integration/e2e`.
- **UNVERIFIED:** the Kotlin snippets are **not yet compiled** (no Android SDK on the build host — carried from [VALIDATION_EVIDENCE.md](#) Issues); compiling
 - running these tests is in Continuation.

6. Integration testing

- **Go [?] Postgres + MinIO via testcontainers-go:** integration tests spin up **real PostgreSQL** and **real MinIO** containers (not mocks), apply the MVP migrations, and exercise the server end-to-end through its HTTP surface with `net/http/httptest`.
 - **Server + DB:** register a device, upload artifact metadata, create a release/deployment, and read back state — asserting the rows written match the schema in [schema.md](#) (12 tables, per [VALIDATION_EVIDENCE.md §2](#)).
 - **Server + Storage (MinIO):** upload an artifact blob and re-fetch it (range-get for large `.zip` if the Storage binding supports it — UNVERIFIED per [submodule_reuse_map.md §5](#)).
 - **Artifact-validator + security + Storage:** push a real artifact through upload → validate → persist, asserting good artifacts pass and bad ones (forged/tampered/downgrade) are rejected at the right stage.
- **Android agent integration (host/emulated A/B):** a **fake/host update_engine bridge** plus a **real ZIP_STORED payload** exercises `download → verify → applyPayload → onStatusUpdate → onPayloadApplicationComplete`, and WorkManager scheduling is driven by a test driver / fake clock to confirm the 15-min + jitter cadence and backoff — exactly the plan recorded in [integration_guide.md](#) (runtime/ integration).
- **Rollout** (*deferred*) — engine integrated against a real database-backed store with injected telemetry signals.

7. Contract testing (OpenAPI conformance)

- **Source of truth:** `openapi.yaml` — OpenAPI 3.1.0, 12 paths, 24 component schemas (confirmed valid by redocly in [VALIDATION_EVIDENCE.md §1](#)).
- **Spec validity (artifact gate):** `npx --yes @redocly/cli@latest lint api/openapi.yaml` runs in CI as a standing gate (already passing, single cosmetic info-license warning).
- **Server-side conformance:** the running Go server's responses are validated against the OpenAPI schema (request/response shape, status codes, required fields) for each of the 12 paths — i.e. the implementation must not drift from the contract. Drift is a test failure.
- **Client-side (consumer) conformance:** the Android agent's expectations of `/client/update` and `/client/telemetry` are checked against the same spec via the shared `ota-protocol` types, so server and agent share one contract.
- **Anti-drift invariant:** `ota-protocol` wire types and `openapi.yaml` describe the same wire contract; a contract test asserts they agree (any mismatch fails CI).

8. End-to-end on the Orange Pi 5 Max

The on-board e2e is the §1 runtime layer's hardware tier on the **Orange Pi 5 Max / RK3588, Android 15** target ([integration_guide.md](#), [build_integration.md](#)). It is **not yet executed**; the board's AOSP A/B + dynamic partitions + Virtual A/B + dm-user enablement is **UNVERIFIED** and is the top-priority hardware gate (carried from [integration_guide.md](#) and [update_engine_integration.md](#)).

8.1 Happy-path runbook

1. **Flash** the Helix-enabled AOSP build (with the `ota-android-agent` as a platform-signed system app per [build_integration.md](#)) to the board.
2. **Upload** a new OTA artifact via the dashboard (server intake → artifact-validator → Storage).
3. **Deploy-to-all** from the dashboard (create a deployment targeting the board).
4. Device **downloads + verifies**: the agent downloads the full zip to `/data/ota_package/`, verifies SHA-256 + signature + the four hashes ([update_engine_integration.md](#)).
5. **A/B apply**: `applyPayload` writes only the **inactive** slot (Virtual A/B: into the COW snapshot via `snapuserd`); `update_engine` re-verifies metadata + payload signature.
6. **Reboot** into the new slot (trial state via `boot_control`).
7. **Post-boot success**: `update_verifier` / agent marks the slot successful; the agent reports a success telemetry event to the server.

8.2 Failure-path runbook (automatic A/B rollback)

1. **Corrupt the new (inactive) slot** after apply (or inject a payload that fails post-boot verification).
2. **Reboot**: the new slot fails its trial; the bootloader/`update_verifier` does **not** mark it successful.

3. **Confirm automatic A/B rollback:** the device falls back to the previous good slot (the running slot was never touched during apply), and the agent reports a failure/rollback telemetry event.

This validates the no-corruption guarantee (A/B + AVB/dm-verity + boot_control auto-rollback) described in [update_engine_integration.md §7](#) and the LOCKED native-Android-A/B strategy ([ADR-0001](#)). The agent **wraps, never replaces** the engine, so rollback is enforced by the bootloader, not by Helix code.

UNVERIFIED: every step in §8 depends on the board actually shipping AOSP A/B + VAB + AVB; until that is confirmed on hardware, the e2e result is unknown.

9. Security testing

Security tests target the trust boundary defined in [signing_verification.md](#), [transport_security.md](#), [threat_model.md](#), and [ADR-0002](#). Each is a negative test that MUST fail closed:

- **Forged artifact** — an artifact whose contents do not match its manifest/hashes is rejected by `ota-artifact-validator` (structure/hash stage) and, defense-in-depth, by `update_engine` on apply.
- **Bad signature** — an artifact with an invalid/absent signature is rejected by the validator (signature stage) on the server and re-checked by `update_engine` (payload signature) on the device.
- **Downgrade / anti-rollback** — pushing an older build to a newer device is rejected: the server-side **version monotonicity** check rejects it at intake, and on-device `update_engine` refuses it with `PAYLOAD_TIMESTAMP_ERROR` (51) per [update_engine_integration.md](#). The rollout policy must never attempt a downgrade.
- **MITM** — transport is exercised against a tampered/intercepted channel to confirm TLS and the on-device verify-before-apply window defeat in-flight modification ([transport_security.md](#)); a body altered in transit must fail hash/signature verification before apply.

These map directly to the artifact-validator and signing components and are run both in integration (server-side) and in the e2e (device-side re-verification).

10. Load testing

- **Tool:** [k6](#).
- **Scenarios:** concurrent **device poll** (`/client/update`) and **artifact download** modeling a fleet polling on the 15-min + jitter cadence ([integration_guide.md](#)), plus concurrent telemetry ingest (`/client/telemetry`).
- **Targets under load:** the **server** (HTTP/3→HTTP/2 transport, `ratelimiter`), the **API** paths, the **DB** (read/write under concurrent poll + telemetry), and artifact **download** throughput from Storage/MinIO (range-get behavior for large .zip — UNVERIFIED whether Storage supports streaming/range-get, per [submodule_reuse_map.md §5](#)).
- **Assertions:** error rate, p95/p99 latency, and that `ratelimiter` throttles abusive clients without starving healthy ones. Concrete SLO thresholds are

UNVERIFIED for this revision and are set in Continuation once a baseline run exists.

11. Mutation testing (meta-test, §1.1)

The mutation meta-test (HelixConstitution §1.1, VERIFIED — “False-positive immunity is an invariant”, plus **Appendix A** “Mutation testing — academic and industrial foundations”) tests the tests: it seeds faults into the code under test and asserts the suite **kills** them (a surviving mutant means the suite has a blind spot).

- **Go modules** (ota-protocol, ota-artifact-validator, ota-rollout-engine [deferred], server handlers, signing helpers): run a Go mutation engine — **gremlins** and/or **go-mutesting** — over the unit suites in §5.1. The validator and protocol are the highest value targets because their unit tests are pure-function table tests where mutation score is most meaningful (e.g. flipping a comparison in version monotonicity must be caught).
- **Kotlin / JVM modules** (the host-testable portions of ota-android-agent, ota-telemetry-schema, ota-update-engine-bridge): run **pitest** over the JVM-target unit suites in §5.2.
- **Scope:** mutation testing runs on the NEW ota-* modules’ own suites only; catalogue submodules are out of scope here (their suites are their own responsibility).
- **Gate:** a per-module **mutation-score threshold** is enforced in CI. The exact threshold is **UNVERIFIED** for this revision (no suite has been run yet) and is fixed in Continuation after a first measured run.
- **Tool availability:** none of gremlins, go-mutesting, or pitest is confirmed installed on the current build host (only the tools listed in VALIDATION_EVIDENCE.md §5 are confirmed present); wiring them into CI is in Continuation. Marked **UNVERIFIED**.

12. Already-performed validation (evidence)

The following are **executed, reproducible** results — the §1 artifact gate (§2) — recorded in VALIDATION_EVIDENCE.md. They are the only confirmed test outcomes in this corpus to date; the CI artifact gate re-runs these exact validators:

Artifact	Validator (real tool)	Result
api/openapi.yaml	npx @redocly/ cli@latest lint	Valid — OpenAPI 3.1.0, 12 paths, 24 schemas; 1 cosmetic info-license warning (§1).
database/migrations/ 001_initial_schema. {up,down}.sql	psql -v ON_ERROR_STOP=1 against a live PostgreSQL server	UP OK (12 tables created), DOWN OK (clean teardown) (§2).
deployment/ kubernetes/*.yaml	kubeconform -summary -strict	5 resources, Valid: 5, Invalid: 0 (§3).
deployment/docker- compose.mvp.yml	YAML parse only (docker engine absent on host)	Parses; full docker compose config deferred to CI (§4).

Carried-forward limitations from that evidence: docker-compose is not yet docker compose config-validated (engine absent), and the Kotlin snippets are **not compiled** (no Android SDK on host) — both are UNVERIFIED and tracked in Continuation here and in VALIDATION_EVIDENCE.md.

13. Anti-bluff notes

Per HelixConstitution §7.1 (“NO BLUFF — positive-evidence-only validation”), §11.4.6 (“No-guessing mandate”) and §11.4.123 (“Rock-solid-proof-or-deep-research mandate”) — all three clause numbers VERIFIED against the live Constitution.md (§7.1 line ~282, §11.4.6 “No-guessing mandate” line ~630, §11.4.123 the 2026-06-03 user mandate line ~8694) — and documentation_standards.md §8:

- **No fabricated results.** The only *executed* test results referenced here are those in VALIDATION_EVIDENCE.md (§12). Every unit/integration/contract/ e2e/ security/load/mutation item is a **plan to be implemented**, not a claimed pass; the six NEW ota-* repos do not yet exist, so no test code has been run against them.
- **Only real modules referenced.** Submodules are named only from the verified catalogue (documentation_standards.md §9) plus the **six NEW** ota-* submodules from submodule_reuse_map.md §4. No submodule name is invented.
- **UNVERIFIED tagged.** The Orange Pi 5 Max board’s A/B/VAB/AVB enablement, the Kotlin snippet compilation, the mutation-tool availability (gremlins/go-mutesting/pitest not confirmed on the build host), the load SLO thresholds, and the Storage range-get capability remain tagged UNVERIFIED inline where used. **Resolved this revision:** the HelixConstitution clause numbers (§1, §1.1, §7.1, §11.4.6, §11.4.61, §11.4.123) are now VERIFIED against the live Constitution.md and are no longer UNVERIFIED — leaving a verifiable fact tagged UNVERIFIED would itself be a §11.4.6 (no- guessing) / §7.1 deviation, so the tags were corrected after reading the source.
- **Open work in Continuation.** Standing up the NEW repos, the CI matrix, the on-board e2e, and the mutation gate all live in the Continuation row, not as completed claims.